

On This Page >

How Templates work

Clan offers the ability to use templates for creating different resources. It comes with some

`<builtin>` templates and discovers all exposed templates from its flake's `inputs`

For example, one can list all current templates like this:

```

1 $ clan templates list
2 Available 'clan' templates
3 |— <builtin>
4 |   |— default: Initialize a new clan flake
5 |   |— flake-parts: Flake-parts
6 |   |— minimal: for clans managed via (G)UI
7 |— inputs.self:
8 |   |— default: Initialize a new clan flake
9 |   |— flake-parts: Flake-parts
10 |   |— minimal: for clans managed via (G)UI
11 Available 'disko' templates
12 |— <builtin>
13 |   |— single-disk: A simple ext4 disk with a single partition
14 |— inputs.self:
15 |   |— single-disk: A simple ext4 disk with a single partition
16 Available 'machine' templates
17 |— <builtin>
18 |   |— demo-template: Demo machine for the CLAN project
19 |   |— flash-installer: Initialize a new flash-installer machine
20 |   |— new-machine: Initialize a new machine
21 |   |— test-morph-template: Morph a machine
22 |— inputs.self:
23 |   |— demo-template: Demo machine for the CLAN project
24 |   |— flash-installer: Initialize a new flash-installer machine
25 |   |— new-machine: Initialize a new machine
26 |   |— test-morph-template: Morph a machine
  
```



Using `<builtin>` Templates

Templates are referenced via the `--template` `selector`

clan-core ships its native/builtin templates. Those are referenced if the selector is a plain string (without `#` or `./.`)

For example:

```
clan flakes create --template=flake-parts
```

would use the native `<builtin>.flake-parts` template

Selectors follow nix flake `reference#attribute` syntax

Selectors follow a very similar pattern as Nix's native attribute selection behavior.

Just like `nix build .` would build `packages.x86-linux.default` of the flake in `./.`

`clan flakes create --template=.` would create a clan from your **local** `default` clan template (`templates.clan.default`).

In fact this command would be equivalent, just make it more explicit

```
clan flakes create --template=.#clan.templates.clan.default (explicit path)
```

Remote templates

Just like with Nix you could specify a remote url or path to the flake containing the template

```
clan flakes create --template=github:owner/repo#foo
```

Implementation Note

Not all features of Nix's attribute selection are currently matched. There are minor differences in case of unexpected behavior please create an [issue](#)

← Previous

Managing macOS Machines with Clan

Next →

Community Disko Templates

Need help? Reach out to [our community](#).